CC232 - Algoritmos y Estructuras de Datos

Simulacro de Examen Parcial - Semanas 1 a 7

Universidad Nacional de Ingeniería - Ciclo 2026-2

Duración	180 minutos	**Tiempo adicional**	Hasta 20 minutos para revisar
Puntaje	28 puntos (7 problemas de 4 puntos)	**Modalidad**	Individual, presencial, lápiz y papel

Nombre: _____ Código: _____

Indicaciones

- Responde en las hojas de respuesta entregadas. Conserva los diagramas necesarios para justificar tu solución.
- No uses computadora, celular, Internet, apuntes ni asistentes de inteligencia artificial.
- No necesitas sintaxis Java perfecta. Puedes usar Java simplificado o pseudocódigo preciso si las operaciones y las referencias quedan claras.
- No basta con indicar una complejidad. Explica qué trabajo depende de n (o de h).
- Antes de modificar una estructura identifica qué información debes conservar. Después comprueba que la representación sigue siendo válida.
- No uses `java.util.ArrayList`, `java.util.LinkedList`, `TreeMap`, `PriorityQueue` ni otras estructuras de Java Collections.
- El solucionario se encuentra al final del documento. Resuelve primero sin mirarlo.

Distribución y tiempo sugerido

Problema	Tema (semana)	Puntaje	Tiempo sugerido
1	Arreglo dinámico: resize, desplazamientos y compactación (S1)	4	25 min
2	Listas enlazadas: inversión en SLList y DLList circular (S2)	4	25 min
3	Pilas y colas: ArrayQueue circular y cola con dos pilas (S3)	4	25 min
4	BST: inserción, recorridos y sucesor (S4)	4	25 min
5	BST: eliminación con `splice` y sucesor (S5)	4	25 min
6	AVL: rotaciones y `rebalance` (S6)	4	30 min
7	BinaryHeap: `heapify`, `remove`, `add` (S7)	4	25 min
	Total	**28**	**180 min**

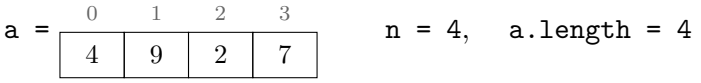
Criterio general. Se evalúa la coherencia entre la representación, el algoritmo, el estado obtenido, los invariantes y la complejidad. Una respuesta con error menor de sintaxis puede recibir crédito si el razonamiento estructural es correcto. Una complejidad sin justificación no constituye una respuesta completa.

Problema 1. Buffer de telemetría sobre arreglo dinámico - 4 puntos

Un módulo de telemetría almacena lecturas en un `ArrayStack` (arreglo dinámico) con campos `Integer[] a` (arreglo de respaldo) y `n` (cantidad de elementos lógicos). La política de redimensionamiento es la trabajada en clase:

- `add(i, x)`: si `n + 1 > a.length`, se ejecuta `resize()` antes de desplazar e insertar.
- `remove(i)`: después de desplazar y decrementar `n`, si `a.length >= 3 * n`, se ejecuta `resize()`.
- `resize()`: crea un arreglo de capacidad `Math.max(1, 2 * n)` y copia `a[0..n-1]`.

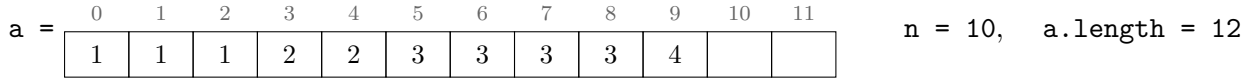
Estado inicial:



Se ejecuta, en este orden: `add(5)` (al final), `add(1, 6)`, `remove(0)`, `remove(4)`, `remove(0)`, `remove(0)`.

a) Construye una tabla que muestre, después de cada operación, el contenido lógico, `n`, `a.length`, la cantidad de elementos desplazados y la cantidad de elementos copiados por `resize()`. Indica el total de desplazamientos, el total de copias y el estado físico final del arreglo. **(1,0 punto)**

Segunda parte. El sensor ahora entrega lecturas ordenadas de forma no decreciente, con muchas repeticiones. Para ahorrar memoria se desea implementar `int keepAtMostTwo()`: la operación conserva como máximo dos copias de cada valor, mantiene el orden relativo, deja los elementos conservados ocupando consecutivamente `a[0..n-1]`, coloca `null` en las posiciones que dejan de pertenecer al rango lógico y retorna la cantidad eliminada. La capacidad no se reduce.



`resultado lógico: [1, 1, 2, 2, 3, 3, 4]` `valor retornado: 3`

b) Diseña un algoritmo que realice una sola pasada principal sobre los elementos originales usando un número constante de variables auxiliares (por ejemplo, `read` y `write`). No crees otro arreglo y no llames repetidamente a `remove(i)`. Escribe Java simplificado o pseudocódigo preciso. **(1,0 punto)**

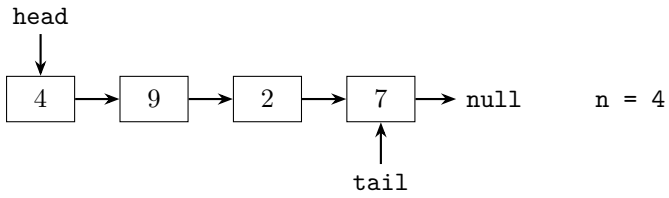
c) Escribe un invariante del ciclo que explique qué representan los primeros `write` elementos después de haber examinado los primeros `read` elementos originales, y por qué una escritura en `a[write]` no destruye información que todavía debe examinarse. **(0,5 puntos)**

d) Un compañero propone conservar `a[read]` cuando `write < 2 || a[read] != a[read - 2]`, es decir, comparar con la posición `read - 2` en lugar de compararla con lo ya escrito. Construye un contraejemplo pequeño, realiza su traza y explica exactamente por qué falla. **(0,75 puntos)**

e) Evalúa la afirmación: *"Como en un arreglo dinámico las operaciones cuestan O(1) amortizado, vaciar el arreglo con n llamadas a `remove(0)` cuesta O(n) en total"*. Indica qué trabajo depende de n. **(0,75 puntos)**

Problema 2. Inversión de una lista de reproducción - 4 puntos

Parte A. Una lista de reproducción se representa con una `SLList` con campos `head`, `tail` y `n`.



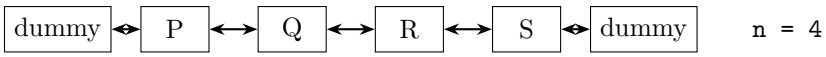
Se desea implementar `void reverse()`, que invierte la lista en el mismo lugar. No crees nodos, no copies valores y no conviertas la lista en un arreglo.

a) En el ciclo de inversión se ejecuta la asignación `cur.next = prev`. Identifica qué referencia debe conservarse *antes* de esa asignación y explica para qué se necesita. Dibuja el estado que se obtendría en la primera iteración si no se conserva. **(0,75 puntos)**

b) Escribe `reverse()` para la `SLList`, incluyendo la actualización de `head` y `tail`. Verifica los casos $n = 0$ y $n = 1$. **(0,75 puntos)**

c) Realiza la traza sobre la lista dada mostrando `prev`, `cur` y la referencia conservada en cada iteración. Dibuja el estado final con `head` y `tail`. **(0,5 puntos)**

Parte B. Otra lista se representa con una `DLList` circular con nodo centinela `dummy`:



(El nodo `dummy` aparece dos veces solo para dibujar la circularidad: `dummy.next` es `P` y `dummy.prev` es `S`.)

Se propone invertir la lista recorriendo *todos* los nodos, incluido `dummy`, e intercambiando en cada uno los campos `prev` y `next`.

d) Escribe el fragmento que realiza la inversión. Explica en qué campo queda el siguiente nodo original después del intercambio, cómo avanzas y cómo sabes que terminaste. Verifica el caso $n = 0$. **(1,0 punto)**

e) Un estudiante intercambia `prev` y `next` solamente en los nodos reales `P`, `Q`, `R`, `S` y no en `dummy`. Describe las referencias resultantes de `dummy`, `P` y `S`, señala un invariante violado e indica qué secuencia se obtendría al recorrer desde `dummy.next` siguiendo `next`. **(0,5 puntos)**

f) Compara el costo temporal y la memoria auxiliar de ambas inversiones. Explica por qué ninguna inversión en el mismo lugar puede costar menos que $O(n)$. **(0,5 puntos)**

Problema 3. Cola de impresión circular y cola con dos pilas - 4 puntos

Parte A. Un servidor de impresión usa una `ArrayQueue` con campos `Integer[] a`, `j` (índice físico del frente lógico) y `n`. Invariante: el elemento lógico k se encuentra en `a[(j + k) % a.length]`, para $0 \le k < n$.

- `add(x)`: si `n + 1 > a.length`, `resize()`; luego `a[(j + n) % a.length] = x; n++`.
- `remove()`: `x = a[j]; a[j] = null; j = (j + 1) % a.length; n--`; si `a.length >= 3 * n`, `resize()`; retorna `x`.
- `resize()`: crea `b` de capacidad `Math.max(1, 2 * n)`, copia `b[k] = a[(j + k) % a.length]` para $0 \le k < n$, y luego `j = 0`.

Estado inicial:

`a =`

0	1	2	3	4	5
30	40	null	null	10	20

`j = 4, n = 4, a.length = 6`

a) Escribe la vista lógica inicial. Luego ejecuta, en este orden: `add(50)`, `remove()`, `add(60)`, `add(70)`, `add(80)`, `remove()`, `remove()`, `remove()`. Construye una tabla con el arreglo físico, `j`, `n`, `a.length` y el valor retornado después de cada operación. **(1,25 puntos)**

b) Un estudiante implementa `resize()` con una copia física (`b[k] = a[k]`, equivalente a `Arrays.copyOf`) y conserva el valor de `j`. Muestra qué ocurre en la operación `add(80)` de la traza anterior y explica por qué se rompe el invariante aunque ningún valor se haya perdido físicamente. **(0,5 puntos)**

c) Escribe los invariantes de `ArrayQueue` y explica por qué `remove()` no necesita desplazar elementos, a diferencia de `remove(0)` en un `ArrayStack`. **(0,5 puntos)**

Parte B. Se desea implementar una cola FIFO usando únicamente dos pilas `LinkedStack` de enteros, `inStack` y `outStack`, con las operaciones `push`, `pop` e `isEmpty`. La idea es: `add(x)` apila en `inStack`; `remove()` desapila de `outStack` y, solo cuando `outStack` está vacía, transfiere todos los elementos de `inStack` a `outStack`.

d) Escribe `add(int x)` y `int remove()` (lanza una excepción si la cola está vacía). Explica por qué el orden FIFO se conserva y qué ocurriría si se transfiriera también cuando `outStack` no está vacía. **(1,0 punto)**

e) Realiza la traza de `add(1)`, `add(2)`, `add(3)`, `remove()`, `add(4)`, `remove()`, `remove()`, mostrando el contenido de ambas pilas (indica el tope) y los valores retornados. Cuenta las operaciones `push` y `pop` realizadas y justifica por qué `remove()` es $O(1)$ amortizado aunque una llamada concreta pueda costar $O(n)$. **(0,75 puntos)**

Problema 4. Índice de identificadores en un BST - 4 puntos

Un sistema de inventario indexa identificadores en un `BinarySearchTree` con `Node {int x; Node left, right, parent;}`, campos `root` y `n`. Se rechazan duplicados. **Convención de altura del curso:** árbol vacío $h = 0$, hoja $h = 1$ (se cuentan nodos).

Se insertan, en este orden, en un árbol vacío:

```
50, 30, 70, 20, 40, 60, 80, 35, 45, 65
```

a) Dibuja el BST final e indica su altura h. Indica el camino de comparaciones que realiza `findLast` al insertar `45` y qué referencias se asignan al nuevo nodo. **(0,75 puntos)**

b) Escribe el recorrido inorder y el recorrido preorder. Explica qué invariante del BST garantiza que el inorder sale ordenado. **(0,75 puntos)**

c) Escribe `Node successor(Node u)` usando las referencias `parent` (sin recorrer todo el árbol). Aplícalo a los nodos `30`, `45`, `65` y `80`, indicando el camino seguido en cada caso. Expresa su costo en función de h. **(1,0 punto)**

d) Un estudiante propone verificar si un árbol es BST comprobando, para cada nodo `u`, solamente que `u.left.x < u.x < u.right.x` (cuando los hijos existen). Construye un contraejemplo de a lo más cuatro nodos que pase su verificación sin ser BST, muestra qué operación pública fallaría y propone una verificación correcta. **(0,75 puntos)**

e) Inserta las claves `15, 25, 35, 45, 55, 65, 75` en el orden dado y, por separado, en el orden `45, 25, 65, 15, 35, 55, 75`. Dibuja ambos árboles, indica sus alturas y cuántas comparaciones realiza `contains(75)` en cada uno. Evalúa la afirmación *"un BST garantiza búsquedas en O(log n)"*. **(0,75 puntos)**

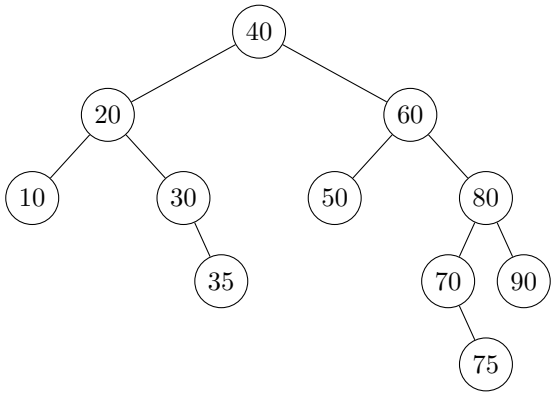
Problema 5. Depuración de un catálogo: eliminación en BST - 4 puntos

Un catálogo se mantiene en un BST con referencias `parent`. La eliminación usa el **sucesor inorder** y la operación auxiliar `splice`:

```
void splice(Node u) {              // precondición: u.left == null || u.right == null
    Node s = (u.left != null) ? u.left : u.right;   // hijo superviviente
    Node p = u.parent;
    if (u == root) root = s;
    else if (p.left == u) p.left = s;
    else p.right = s;
    if (s != null) s.parent = p;
}

boolean remove(int x) {
    Node u = findLast(x);
    if (u == null || u.x != x) return false;
    if (u.left == null || u.right == null) splice(u);
    else {
        Node w = u.right;
        while (w.left != null) w = w.left;   // sucesor inorder
        u.x = w.x;
        splice(w);
    }
    n--;
    return true;
}
```

Estado inicial ($n = 11$):



a) Ejecuta, en este orden, `remove(10)`, `remove(30)`, `remove(60)`, `remove(40)`. Para cada llamada indica el caso (hoja, un hijo, dos hijos), el nodo que se elimina físicamente y dibuja el árbol resultante. **(1,5 puntos)**

b) Para `remove(60)`, enumera en orden cada campo o referencia que se modifica (incluyendo `parent`). Distingue la eliminación lógica de la eliminación física. Explica por qué el sucesor nunca tiene hijo izquierdo, pero sí puede tener hijo derecho, y por qué eso importa en esta llamada. **(0,75 puntos)**

c) Un estudiante omite la última línea de `splice` (`if (s != null) s.parent = p;`). Considera que ese `splice` defectuoso se usa en `remove(30)`. ¿El recorrido inorder posterior es correcto? Indica qué invariante se rompe y describe qué ocurre si después se ejecuta `remove(35)`. **(0,75 puntos)**

d) Separa el costo de localización y el costo de modificación de `remove(x)` en función de h. Luego realiza la traza de `remove(45)` sobre el árbol final de la parte a): camino recorrido, valor retornado y valor de `n`. Explica por qué no se puede afirmar que `remove` sea $O(\log n)$ en un BST. **(1,0 punto)**

Problema 6. Índice balanceado AVL - 4 puntos

Un índice se implementa con un árbol AVL: `Node {int x; int height = 1; Node left, right;}`, sin referencia `parent`. Se usa `height(null) = 0` y

$$\texttt{balanceFactor(u)} = \texttt{height(u.left)} - \texttt{height(u.right)}.$$

Invariante AVL: $|\texttt{balanceFactor(u)}| \leq 1$ para todo nodo. Las operaciones de rebalanceo son:

```
Node rotateRight(Node y) {
    Node x = y.left;
    Node middle = x.right;
    x.right = y;
    y.left = middle;
    updateHeight(y);          // primero el nodo que quedó más abajo
    updateHeight(x);
    return x;                 // nueva raíz local
}
// rotateLeft(Node x) es simétrica: y = x.right; middle = y.left; ...

Node rebalance(Node u) {
    int bf = balanceFactor(u);
    if (bf > 1) {
        if (balanceFactor(u.left) < 0) u.left = rotateLeft(u.left);
        return rotateRight(u);
    }
    if (bf < -1) {
        if (balanceFactor(u.right) > 0) u.right = rotateRight(u.right);
        return rotateLeft(u);
    }
    return u;
}
```

Se insertan, en este orden, en un AVL vacío:

`10, 20, 30, 25, 27, 5, 4, 35, 33`

a) Para cada inserción que produce desbalance indica el nodo donde se detecta $|\texttt{balanceFactor}| = 2$ (el primero desde abajo), el caso (LL, RR, LR o RL) y las rotaciones aplicadas. Dibuja el subárbol antes y después de rebalancear y dibuja el árbol final. **(1,5 puntos)**

b) Para la inserción de `27`, escribe la secuencia de referencias que modifican `rotateLeft` y `rotateRight`, y el orden de las llamadas a `updateHeight`. Explica qué altura incorrecta se obtendría si en `rotateRight` se actualizara `x` antes que `y`. **(0,75 puntos)**

c) Un estudiante escribe `rebalance` sin la comprobación `balanceFactor(u.left) < 0`, es decir, cuando `bf > 1` siempre ejecuta solamente `rotateRight(u)`. Aplica su versión a la inserción de `27` y demuestra, con los factores de balance, que el resultado no es un AVL. **(0,75 puntos)**

d) Construye una tabla con `height` y `balanceFactor` de cada nodo del árbol final. **(0,5 puntos)**

e) Sea $N(h)$ la mínima cantidad de nodos de un AVL de altura h, con $N(1) = 1$, $N(2) = 2$ y $N(h) = 1 + N(h-1) + N(h-2)$. Calcula $N(h)$ hasta $h = 5$ y deduce la altura máxima posible de un AVL con 9 nodos. Compárala con la altura máxima de un BST sin balanceo con 9 nodos. **(0,5 puntos)**

Problema 7. Planificador de tareas con BinaryHeap - 4 puntos

Un planificador atiende tareas por prioridad usando un `BinaryHeap` (min-heap: menor valor, mayor prioridad) con campos `Integer[] a` y `n`. Índices desde 0:

$$\texttt{left(i)} = 2i + 1, \qquad \texttt{right(i)} = 2i + 2, \qquad \texttt{parent(i)} = \lfloor (i-1)/2 \rfloor.$$

Invariante: `a[parent(i)] <= a[i]` para todo $0 < i < n$, y los nodos ocupan exactamente `a[0..n-1]`.

- `add(x)`: `a[n++] = x; bubbleUp(n - 1)`.
- `remove()`: `min = a[0]; a[0] = a[--n]; a[n] = null; trickleDown(0)`; retorna `min`.
- `trickleDown(i)`: intercambia con el *menor hijo válido* mientras ese hijo sea menor que `a[i]`.
- `heapify()`: ejecuta `trickleDown(i)` para `i = n/2 - 1` hasta `0`.

Las prioridades llegan en este arreglo:

`a =`

0	1	2	3	4	5	6	7	8
9	4	7	1	8	2	6	3	5

`n = 9`

a) Ejecuta `heapify()`. Para cada valor de `i` muestra los intercambios y el arreglo resultante. Dibuja el heap final como árbol y cuenta el total de intercambios. **(1,25 puntos)**

b) Sobre el heap obtenido en a), ejecuta `remove()`. Indica el valor retornado, cada paso de `trickleDown` y el arreglo final. **(0,75 puntos)**

c) Sobre el resultado de b), ejecuta `add(0)` (supón capacidad suficiente). Indica cada paso de `bubbleUp` y el arreglo final. **(0,5 puntos)**

d) Justifica por qué `heapify()` cuesta $O(n)$, mientras que insertar los n elementos uno por uno con `add` cuesta $O(n \log n)$ en el peor caso. **(0,5 puntos)**

e) Un estudiante implementa `trickleDown` de modo que siempre intercambia con el hijo izquierdo cuando este es menor que el padre. Usa el estado de b) justo después de mover el último elemento a la raíz para mostrar que su versión rompe el invariante. **(0,5 puntos)**

f) Evalúa la afirmación: *"Como el heap es un árbol con altura $O(\log n)$, buscar si una prioridad arbitraria x está en el heap cuesta $O(\log n)$, igual que en un BST balanceado".* **(0,5 puntos)**

Fin del examen. Revisa que tus diagramas, contratos, invariantes y complejidades sean coherentes con la representación que estás analizando.

Solucionario detallado

Criterios de corrección por subpregunta

Cada solución presenta el razonamiento esperado y, al final de cada problema, un criterio de corrección con la distribución del puntaje. Pueden existir redacciones distintas igualmente válidas si conservan los mismos invariantes y la misma justificación de costo.

Solución del Problema 1

a) Traza completa de las operaciones.

Operación	Retorna	Contenido lógico	n	a.length	Desplaz.	Copias
inicial	–	[4, 9, 2, 7]	4	4	–	–
`add(5)`	–	[4, 9, 2, 7, 5]	5	8	0	4
`add(1, 6)`	–	[4, 6, 9, 2, 7, 5]	6	8	4	0
`remove(0)`	4	[6, 9, 2, 7, 5]	5	8	5	0
`remove(4)`	5	[6, 9, 2, 7]	4	8	0	0
`remove(0)`	6	[9, 2, 7]	3	8	3	0
`remove(0)`	9	[2, 7]	2	4	2	2
Total					**14**	**6**

Justificación de cada paso:

- `add(5)`: $n + 1 = 5 > 4$, por lo tanto primero `resize()` a $\max(1, 8) = 8$ copiando 4 elementos. Como se inserta en $i = n = 4$, se desplazan $n - i = 0$ elementos.
- `add(1, 6)`: $6 \leq 8$, no hay resize. Se desplazan hacia la derecha las posiciones 1 a 4, es decir $n - i = 5 - 1 = 4$ elementos.
- `remove(0)`: se desplazan hacia la izquierda $n - i - 1 = 5$ elementos. Luego $n = 5$ y $8 \geq 15$ es falso.
- `remove(4)`: es el último, 0 desplazamientos. $n = 4$ y $8 \geq 12$ es falso.
- `remove(0)`: 3 desplazamientos. $n = 3$ y $8 \geq 9$ es falso.
- `remove(0)`: 2 desplazamientos. $n = 2$ y $8 \geq 6$ es verdadero: `resize()` a $\max(1, 4) = 4$, copiando 2 elementos.

Estado físico final: `a =`

0	1	2	3
2	7	null	null

`n = 2`, `a.length = 4`.

b) Algoritmo con dos índices. Como la secuencia está ordenada, las copias de un mismo valor son contiguas. Si `a[read]` es igual a `a[write - 2]`, entonces también es igual a `a[write - 1]` (que está entre ambos en orden), de modo que ya se conservaron dos copias de ese valor.

```
int keepAtMostTwo() {
    int write = 0;
    for (int read = 0; read < n; read++) {
        // se conserva si todavía no hay dos copias de este valor en el prefijo escrito
        if (write < 2 || !a[read].equals(a[write - 2])) {
            a[write] = a[read];
            write++;
        }
    }
    for (int k = write; k < n; k++) a[k] = null;   // limpiar el rango descartado
    int removed = n - write;
```

```
    n = write;
    return removed;
}
```

Traza resumida sobre `[1,1,1,2,2,3,3,3,3,4]`:

`read`	`a[read]`	`a[write-2]`	¿Conserva?	`write` después	Prefijo `a[0..write-1]`
0	1	–	sí (`write < 2`)	1	[1]
1	1	–	sí (`write < 2`)	2	[1, 1]
2	1	1	no	2	[1, 1]
3	2	1	sí	3	[1, 1, 2]
4	2	1	sí	4	[1, 1, 2, 2]
5	3	2	sí	5	[1, 1, 2, 2, 3]
6	3	2	sí	6	[1, 1, 2, 2, 3, 3]
7	3	3	no	6	[1, 1, 2, 2, 3, 3]
8	3	3	no	6	[1, 1, 2, 2, 3, 3]
9	4	3	sí	7	[1, 1, 2, 2, 3, 3, 4]

Estado final: `a =`

0	1	2	3	4	5	6	7	8	9	10	11
1	1	2	2	3	3	4	null	null	null		

`n = 7`, `a.length = 12`, retorna `3`.

Costo: la pasada principal examina cada elemento una vez, $O(n)$; la limpieza recorre a lo más n posiciones, $O(n)$. Total $O(n)$ con memoria auxiliar $O(1)$.

c) Invariante (al inicio de cada iteración con índice `read`):

1. `a[0..write-1]` contiene, en el mismo orden, los elementos de la secuencia original `a[0..read-1]` conservando como máximo dos copias de cada valor (y por lo tanto está ordenado).
2. $0 \leq$ `write` $\leq$ `read`, de modo que `a[read..n-1]` conserva todavía los valores originales.

Como `write <= read`, la escritura en `a[write]` solo afecta una posición ya examinada (o la misma que se está leyendo), nunca una posición pendiente. Al terminar, `read = n` y el prefijo es exactamente el resultado.

d) Contraejemplo: `a = [1, 1, 1, 2, 2]`, $n = 5$. Resultado esperado: `[1, 1, 2, 2]`.

`read`	`a[read]`	`a[read-2]` (valor actual)	¿Conserva?	Arreglo después
0	1	–	sí	[1, 1, 1, 2, 2], `write = 1`
1	1	–	sí	[1, 1, 1, 2, 2], `write = 2`
2	1	`a[0]` = 1	no	[1, 1, 1, 2, 2], `write = 2`
3	2	`a[1]` = 1	sí	[1, 1, **2**, 2, 2], `write = 3`
4	2	`a[2]` = **2**	no	[1, 1, 2, 2, 2], `write = 3`

Resultado obtenido: `[1, 1, 2]`; se perdió una copia de `2`. El criterio "`a[read]` distinto del original en `read-2`" sería correcto sobre la secuencia original, pero la celda `read - 2` pertenece a la zona que ya pudo ser sobrescrita (en la iteración 3 se escribió `2` en la posición 2, que originalmente contenía `1`). La comparación correcta debe hacerse contra lo ya conservado, `a[write - 2]`, porque la pregunta real es "¿cuántas copias de este valor ya conservé?".

e) La afirmación es falsa. El $O(1)$ amortizado describe el costo de `resize()` repartido entre muchas operaciones; no elimina el costo de desplazamiento. `remove(i)` desplaza $n - i - 1$ elementos, y `remove(0)` desplaza todos los restantes:

$$(n-1) + (n-2) + \cdots + 1 + 0 = \frac{n(n-1)}{2} \in O(n^2).$$

Solo `remove(n - 1)` (eliminar al final) es $O(1)$ amortizado. Los redimensionamientos por reducción suman a lo más $O(n)$ en total y no cambian la conclusión.

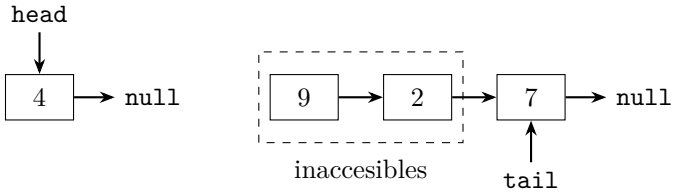
Criterio de corrección (4 puntos).

- a) 1,0: crecimiento a 8 con 4 copias (0,25); desplazamientos correctos (0,25); reducción detectada con $a.length \geq 3n$ en el último paso (0,25); totales y estado físico con `null` (0,25).
- b) 1,0: condición correcta contra `a[write-2]` (0,5); una sola pasada con $O(1)$ memoria (0,25); limpieza a `null`, actualización de `n` y retorno (0,25).
- c) 0,5: significado del prefijo (0,25); `write <= read` y no destrucción de datos pendientes (0,25).
- d) 0,75: contraejemplo válido (0,25); traza que muestra la sobrescritura (0,25); explicación de la causa (0,25).
- e) 0,75: distinguir resize amortizado de desplazamiento (0,25); suma explícita (0,25); conclusión $O(n^2)$ (0,25).

Solución del Problema 2

a) Antes de `cur.next = prev` se debe conservar `nxt = cur.next`. En una `SLList` el campo `cur.next` es la *única* referencia al resto de la lista: no existe `prev` en los nodos. Si se sobrescribe sin guardarla, el resto deja de ser alcanzable.

Primera iteración sin conservarla (`prev = null`, `cur = 4`): se ejecuta `4.next = null` y no hay forma de avanzar.



Los nodos `9` y `2` quedan inalcanzables. El nodo `7` solo es alcanzable por `tail`, pero desde él no se puede retroceder. Se viola el invariante "desde `head`, siguiendo `next`, hay exactamente n nodos".

b)

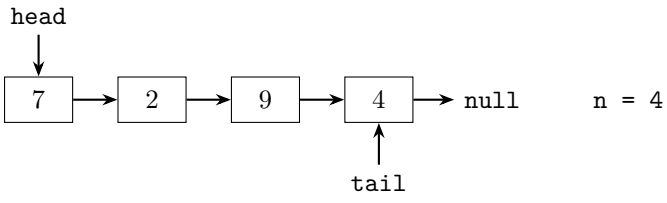
```
void reverse() {
    Node prev = null;
    Node cur = head;
    tail = head;                  // el primer nodo será el último
    while (cur != null) {
        Node nxt = cur.next;      // conservar el resto de la lista
        cur.next = prev;          // invertir un enlace
        prev = cur;               // crece el prefijo invertido
        cur = nxt;                // avanzar sobre la parte no procesada
    }
    head = prev;                  // n no cambia
}
```

Invariante del ciclo: `prev` es la cabeza de la parte ya invertida, `cur` es la cabeza de la parte aún no procesada, ambas partes son disjuntas y juntas contienen los n nodos.

Casos borde: si $n = 0$, `head = null`, el ciclo no se ejecuta y `head = tail = null`. Si $n = 1$, una iteración deja `x.next = null`, `head = tail = x`.

c) Traza:

Iteración	`prev` (antes)	`cur` (antes)	`nxt`	Enlace invertido	`prev`, `cur` (después)
1	null	4	9	`4.next = null`	4, 9
2	4	9	2	`9.next = 4`	9, 2
3	9	2	7	`2.next = 9`	2, 7
4	2	7	null	`7.next = 2`	7, null



Postcondición: `tail.next == null` y la secuencia lógica es la inversa de la original.

d)

```
Node u = dummy;
do {
    Node t = u.next;      // siguiente en el orden original
    u.next = u.prev;      // intercambio de enlaces
    u.prev = t;
    u = t;                // equivalente a u = u.prev después del intercambio
} while (u != dummy);
```

Después del intercambio, el siguiente nodo original queda en `u.prev`; por eso se guarda en `t` y se avanza con `u = t`. Se visitan los $n+1$ nodos (incluido `dummy`) y se termina al volver a `dummy`. Se usa `do-while` porque el recorrido empieza precisamente en `dummy`.

Iteración	`u`	`u.next` después	`u.prev` después
1	dummy	S	P
2	P	dummy	Q
3	Q	P	R
4	R	Q	S
5	S	R	dummy

Resultado: `dummy <-> S <-> R <-> Q <-> P <-> dummy`. Caso $n = 0$: `dummy.next = dummy.prev = dummy`; una iteración intercambia dos referencias iguales, `u` vuelve a `dummy` y termina.

e) Sin intercambiar en `dummy`:

- `dummy.next = P`, `dummy.prev = S` (sin cambio).
- `P.next = dummy`, `P.prev = Q`.
- `S.next = R`, `S.prev = dummy`.

Se viola `u.next.prev == u` para `u = dummy`: `dummy.next.prev = P.prev = Q`, distinto de `dummy`. Al recorrer desde `dummy.next` se visita `P`, luego `P.next = dummy` y el recorrido termina: se obtiene `[P]` aunque $n = 4$. Recorriendo hacia atrás desde `dummy.prev` se obtiene solo `[S]`.

f) Ambas inversiones cuestan $O(n)$: cada nodo se visita una vez y en cada uno se realiza una cantidad constante de asignaciones (la DLList visita $n+1$ nodos por el centinela). No hay costo de localización separado: el trabajo es recorrer. La memoria auxiliar es $O(1)$ (tres referencias en la SLList, dos en la DLList). No puede ser mejor que $O(n)$ porque el enlace de cada uno de los n nodos debe cambiar, y cada cambio es al menos una asignación.

Criterio de corrección (4 puntos).

- a) 0,75: identificar `cur.next` (0,25); explicar que es la única referencia al resto (0,25); dibujo del estado roto (0,25).
- b) 0,75: ciclo correcto (0,5); actualización de `head` y `tail` con casos borde (0,25).
- c) 0,5: tabla de traza (0,25); estado final con `head`, `tail` y `null` (0,25).
- d) 1,0: intercambio correcto (0,25); avance usando el siguiente original (0,25); visita de los $n+1$ nodos con condición de parada (0,25); caso $n = 0$ (0,25).
- e) 0,5: referencias resultantes e invariante violado (0,25); recorrido `[P]` (0,25).
- f) 0,5: costo y memoria (0,25); cota inferior $\Omega(n)$ justificada (0,25).

Solución del Problema 3

a) Vista lógica inicial: elemento k en `a[(4 + k) % 6]`: $k = 0 \to$ `a[4] = 10`, $k = 1 \to$ `a[5] = 20`, $k = 2 \to$ `a[0] = 30`, $k = 3 \to$ `a[1] = 40`. Vista lógica: `[10, 20, 30, 40]`.

Operación	Arreglo físico		j	n	a.length	Retorna
inicial	[30, 40, -, -, 10, 20]		4	4	6	–
`add(50)`	[30, 40, 50, -, 10, 20]	pos $(4+4)\,\%6 = 2$	4	5	6	–
`remove()`	[30, 40, 50, -, -, 20]		5	4	6	10
`add(60)`	[30, 40, 50, 60, -, 20]	pos $(5+4)\,\%6 = 3$	5	5	6	–
`add(70)`	[30, 40, 50, 60, 70, 20]	pos $(5+5)\,\%6 = 4$	5	6	6	–
`add(80)`	[20, 30, 40, 50, 60, 70, 80, -, -, -, -, -]		0	7	12	–
`remove()`	[-, 30, 40, 50, 60, 70, 80, -, -, -, -, -]		1	6	12	20
`remove()`	[-, -, 40, 50, 60, 70, 80, -, -, -, -, -]		2	5	12	30
`remove()`	[50, 60, 70, 80, -, -, -, -]		0	4	8	40

(El símbolo "-" representa `null`.)

Detalles clave:

- `remove()` que retorna 10: $n = 4$ y $6 \geq 12$ es falso, no se reduce.
- `add(80)`: $n + 1 = 7 > 6$, `resize()` a 12 copiando en orden lógico `b[k] = a[(5 + k) % 6]`: 20, 30, 40, 50, 60, 70; luego `j = 0`. Se inserta en $(0 + 6)\,\%12 = 6$.
- Último `remove()`: $n = 4$ y $12 \geq 12$ es verdadero; `resize()` a máx$(1, 8) = 8$ copiando `a[(3 + k) % 12]`: 50, 60, 70, 80; `j = 0`.

Vista lógica final: `[50, 60, 70, 80]`.

b) Antes de `add(80)`: `a = [30, 40, 50, 60, 70, 20]`, `j = 5`, $n = 6$. Con copia física: `b = [30, 40, 50, 60, 70, 20, -, -, -, -, -, -]` y `j = 5`. Luego `add(80)` escribe en $(5 + 6)\,\%12 = 11$.

Vista lógica según el invariante: $k = 0 \to$ `b[5] = 20`, $k = 1 \to$ `b[6] = null`, ..., $k = 5 \to$ `b[10] = null`, $k = 6 \to$ `b[11] = 80`. Resultado: `[20, null, null, null, null, null, 80]`.

Los valores 30 a 70 siguen físicamente en `b[0..4]`, pero ya no son alcanzables por la fórmula. La razón es que la circularidad depende de `a.length`: con longitud 6, después del índice 5 se volvía al índice 0; con longitud 12, después del índice 5 sigue el 6. Por eso `resize()` debe copiar en orden lógico y normalizar `j = 0`.

c) Invariantes:

1. $0 \leq$ `j` $<$ `a.length` y $0 \leq n \leq$ `a.length`.
2. El elemento lógico k está en `a[(j + k) % a.length]` para $0 \leq k < n$.
3. Las celdas fuera del rango lógico contienen `null`.

`remove()` solo lee `a[j]`, lo anula y avanza `j`: el nuevo frente pasa a ser el elemento que antes era lógico 1, sin mover ningún valor. Su costo es $O(1)$ (más el resize amortizado). En un `ArrayStack`, el frente está fijo en el índice 0, por lo que `remove(0)` debe desplazar $n - 1$ elementos: $O(n)$.

d)

```
class QueueWithTwoStacks {
    LinkedStack inStack = new LinkedStack();
    LinkedStack outStack = new LinkedStack();

    void add(int x) {
        inStack.push(x);                          // O(1)
    }
```

```
    int remove() {
        if (outStack.isEmpty()) {
            while (!inStack.isEmpty()) {
                outStack.push(inStack.pop());    // la transferencia invierte el orden
            }
        }
        if (outStack.isEmpty()) throw new NoSuchElementException("cola vacía");
        return outStack.pop();
    }
}
```

Invariante: la cola lógica es el contenido de `outStack` de tope a fondo, seguido del contenido de `inStack` de fondo a tope. En `inStack` el tope es el más reciente; al transferir se invierte el orden y el tope de `outStack` queda siendo el más antiguo, es decir, el primero en salir (FIFO).

Si se transfiriera con `outStack` no vacía, elementos más recientes quedarían encima de elementos más antiguos y saldrían antes que ellos. Ejemplo: `outStack` con 2 en el tope y `inStack = [3]`; transferir pondría 3 encima de 2 y `remove()` retornaría 3 antes que 2, violando FIFO.

e) Traza (el tope se escribe a la derecha):

Operación	**inStack**	**outStack**	**Retorna**
`add(1)`	[1]	[]	–
`add(2)`	[1, 2]	[]	–
`add(3)`	[1, 2, 3]	[]	–
`remove()`	[]	[3, 2]	1 (transfirió 3, 2, 1; luego pop)
`add(4)`	[4]	[3, 2]	–
`remove()`	[4]	[3]	2
`remove()`	[4]	[]	3

Conteo: `push` = 4 (en `inStack`) + 3 (en `outStack`) = 7; `pop` = 3 (de `inStack`) + 3 (de `outStack`) = 6. Total: 13 operaciones primitivas para 7 operaciones de cola.

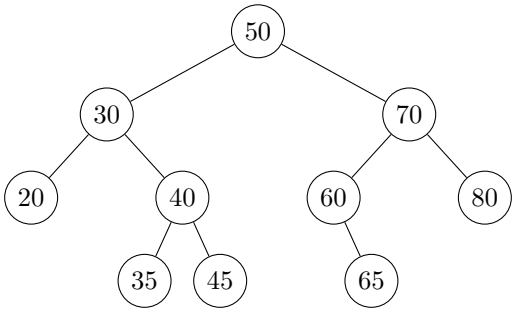
Costo amortizado: cada elemento, durante toda su vida, entra una vez a `inStack`, sale una vez de ella, entra una vez a `outStack` y sale una vez de ella: a lo más 4 operaciones $O(1)$. Una secuencia de m operaciones cuesta $O(m)$ en total, por lo tanto $O(1)$ amortizado por operación. Una llamada concreta a `remove()` puede costar $O(n)$ cuando transfiere n elementos, pero eso ocurre solo después de n inserciones baratas.

Criterio de corrección (4 puntos).

- a) 1,25: vista lógica inicial (0,25); posiciones con módulo correctas (0,25); resize en `add(80)` con copia lógica y `j = 0` (0,25); retornos y `null` (0,25); reducción final a 8 (0,25).
- b) 0,5: estado erróneo con escritura en 11 (0,25); explicación de la dependencia del módulo respecto de `a.length` (0,25).
- c) 0,5: invariantes (0,25); comparación con `remove(0)` de `ArrayStack` (0,25).
- d) 1,0: código correcto con excepción (0,5); justificación FIFO (0,25); contraejemplo de transferencia indebida (0,25).
- e) 0,75: traza correcta (0,25); conteo (0,25); argumento amortizado (0,25).

Solución del Problema 4

a) Árbol final:



Altura $h = 4$ (por ejemplo, el camino 50, 30, 40, 35 tiene 4 nodos).

Inserción de 45: `findLast(45)` compara con 50 ($45 < 50$, va a la izquierda), con 30 ($45 > 30$, derecha), con 40 ($45 > 40$, derecha) y encuentra `null`. Retorna `p = 40`. Se crea `u` con `u.x = 45`, `u.parent = 40` y se asigna `40.right = u`; `n` pasa de 8 a 9.

b) Inorder: `20, 30, 35, 40, 45, 50, 60, 65, 70, 80`.
Preorder: `50, 30, 20, 40, 35, 45, 70, 60, 65, 80`.

Invariante del BST: para todo nodo `u`, todas las claves del subárbol izquierdo son menores que `u.x` y todas las del subárbol derecho son mayores. El inorder visita subárbol izquierdo, luego `u`, luego subárbol derecho; por inducción sobre la altura, cada subárbol se imprime ordenado y `u.x` queda entre ambos, de modo que la salida completa está ordenada.

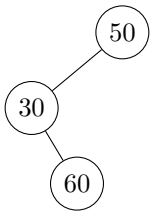
c)

```
Node successor(Node u) {
    if (u.right != null) {             // caso 1: mínimo del subárbol derecho
        Node w = u.right;
        while (w.left != null) w = w.left;
        return w;
    }
    Node p = u.parent;                 // caso 2: subir mientras u sea hijo derecho
    while (p != null && u == p.right) {
        u = p;
        p = p.parent;
    }
    return p;                          // null si u era el máximo
}
```

- `successor(30)`: tiene hijo derecho 40; se baja a la izquierda hasta 35. Resultado: **35**.
- `successor(45)`: no tiene hijo derecho. 45 es hijo derecho de 40, se sube; 40 es hijo derecho de 30, se sube; 30 es hijo izquierdo de 50, se detiene. Resultado: **50**.
- `successor(65)`: no tiene hijo derecho. 65 es hijo derecho de 60, se sube; 60 es hijo izquierdo de 70. Resultado: **70**.
- `successor(80)`: 80 es hijo derecho de 70, se sube; 70 es hijo derecho de 50, se sube; `p = null`. Resultado: **null** (80 es el máximo).

Costo: en el caso 1 se baja por un camino y en el caso 2 se sube por un camino; ambos tienen a lo más h nodos. Costo $O(h)$.

d) Contraejemplo:



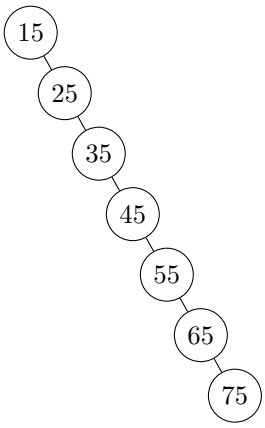
Verificación local: en 50, `30 < 50`; en 30, `30 < 60`. Pasa. Pero 60 está en el subárbol izquierdo de 50 siendo mayor que 50: no es BST. La operación `contains(60)` compara con 50, va a la derecha, encuentra `null` y retorna `false`, aunque 60 está en el árbol.

Verificación correcta con intervalos:

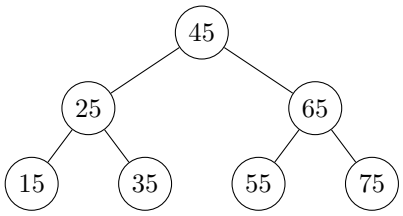
```
boolean isBST(Node u, long low, long high) {
    if (u == null) return true;
    if (u.x <= low || u.x >= high) return false;
    return isBST(u.left, low, u.x) && isBST(u.right, u.x, high);
}
// llamada inicial: isBST(root, Long.MIN_VALUE, Long.MAX_VALUE)
```

Alternativa válida: comprobar que el inorder es estrictamente creciente. Ambas cuestan $O(n)$.

e)



Orden creciente: $h = 7$



Orden balanceado: $h = 3$

`contains(75)`: en el árbol degenerado compara con los 7 nodos (15, 25, . . . , 75); en el balanceado compara con 45, 65 y 75: 3 comparaciones.

La afirmación es falsa: las operaciones de un BST cuestan $O(h)$ y h depende del orden de inserción; puede ir desde $\lceil \log_2(n+1) \rceil$ hasta n. Un BST sin mecanismo de balanceo no garantiza $O(\log n)$; en el peor caso es $O(n)$. Esto motiva los árboles AVL.

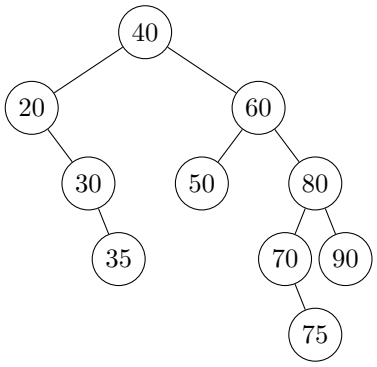
Criterio de corrección (4 puntos).

- a) 0,75: árbol correcto (0,25); altura con la convención del curso (0,25); camino y referencias de 45, incluyendo `parent` (0,25).
- b) 0,75: inorder (0,25); preorder (0,25); justificación con el invariante (0,25).
- c) 1,0: código con ambos casos (0,5); los cuatro resultados con caminos (0,25); costo $O(h)$ (0,25).
- d) 0,75: contraejemplo (0,25); operación que falla (0,25); verificación correcta (0,25).
- e) 0,75: ambos árboles y alturas (0,25); comparaciones (0,25); evaluación de la afirmación (0,25).

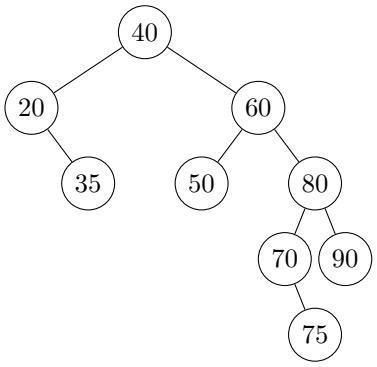
Solución del Problema 5

a) Estados sucesivos.

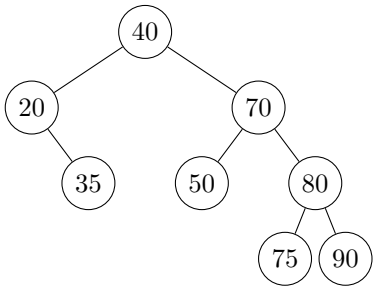
- `remove(10)`: **hoja**. `splice(10)` con `s = null`: `20.left = null`. Se elimina físicamente el nodo 10. $n = 10$.
- `remove(30)`: **un hijo** (35). `splice(30)` con `s = 35`, `p = 20`: `20.right = 35`, `35.parent = 20`. Se elimina físicamente el nodo 30. $n = 9$.
- `remove(60)`: **dos hijos**. Sucesor: `w = 60.right = 80`, se baja a la izquierda a 70, `70.left = null`, entonces `w = 70`. Se copia `u.x = 70` y `splice(70)` con `s = 75`, `p = 80`: `80.left = 75`, `75.parent = 80`. Se elimina físicamente el nodo que contenía 70. $n = 8$.
- `remove(40)`: **dos hijos**. Sucesor: `w = root.right` (nodo con 70), se baja a la izquierda a 50, `50.left = null`, entonces `w = 50`. Se copia `root.x = 50` y `splice(50)` con `s = null`: el nodo con 70 queda con `left = null`. Se elimina físicamente el nodo que contenía 50. $n = 7$.



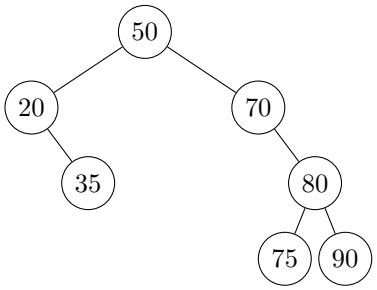
Después de `remove(10)`



Después de `remove(30)`



Después de `remove(60)`



Después de `remove(40)`

Inorder final: `20, 35, 50, 70, 75, 80, 90`, con $n = 7$.

b) Modificaciones de `remove(60)`, en orden:

1. `u.x = 70`: el nodo que ocupaba la posición de 60 ahora almacena 70. Esta es la **eliminación lógica** de 60: el valor desaparece, pero el nodo sigue en el árbol.
2. En `splice(w)` con `w` el nodo original de 70: `s = w.right = 75`, `p = w.parent = 80`; como `p.left == w`, se asigna `80.left = 75`.
3. `75.parent = 80`.
4. En `remove`: `n--` una sola vez.

El nodo `w` deja de ser alcanzable: esa es la **eliminación física**. Los nodos 40, 50 y 90 no cambian.

El sucesor es el mínimo del subárbol derecho de `u`. Si tuviera hijo izquierdo, ese hijo sería menor y también estaría en el subárbol derecho de `u`, contradiciendo que `w` es el mínimo. En cambio, sí puede

tener hijo derecho (aquí 75). Por eso `splice` debe reconectar a 75 con 80; si se hiciera simplemente `80.left = null`, se perdería el subárbol de 75.

c) Con el `splice` defectuoso, `remove(30)` asigna `20.right = 35` pero `35.parent` sigue apuntando al nodo 30, que ya está desconectado.

El inorder posterior es correcto (`20, 35, 40, ...`) porque solo usa `left` y `right`. Una salida visible correcta no implica una representación interna correcta. Se rompe el invariante de consistencia bidireccional: para todo nodo `v` distinto de la raíz, `v.parent.left == v` o `v.parent.right == v`. Aquí `35.parent.right == 35` se cumple sobre un nodo que ya no pertenece al árbol.

Si después se ejecuta `remove(35)`: `findLast` llega a 35 por `20.right`; `splice(35)` toma `p = 35.parent` (el nodo 30 desconectado) y asigna `p.right = null` sobre ese nodo fantasma. El nodo 20 sigue apuntando a 35, así que 35 permanece en el árbol, pero `n` se decrementa. Resultado: `contains(35)` sigue siendo verdadero y `n` ya no coincide con la cantidad de nodos alcanzables.

d) Costo de `remove(x)`:

- **Localización**: `findLast(x)` baja por un camino desde la raíz: $O(h)$. Si `u` tiene dos hijos, la búsqueda del sucesor continúa bajando desde `u`; el camino total sigue teniendo a lo más h nodos: $O(h)$.
- **Modificación**: la copia `u.x = w.x` y `splice` modifican una cantidad constante de referencias (a lo más tres): $O(1)$.
- Total: $O(h)$; la localización domina el costo.

Traza de `remove(45)` sobre el árbol final: $45 < 50$, izquierda a 20; $45 > 20$, derecha a 35; $45 > 35$, derecha `null`. `findLast` retorna el nodo 35; como `35 != 45`, `remove` retorna `false`. No se modifica ninguna referencia y `n` sigue en 7.

No se puede afirmar $O(\log n)$ porque un BST sin balanceo puede tener $h = n$ (por ejemplo, si las claves se insertaron ordenadas), en cuyo caso `remove` cuesta $O(n)$.

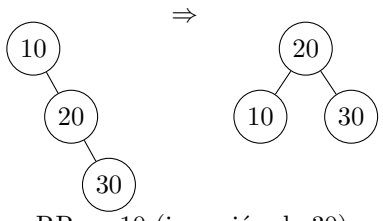
Criterio de corrección (4 puntos).

- a) 1,5: cada eliminación con caso, nodo físico eliminado y árbol correcto (0,375 cada una).
- b) 0,75: lista ordenada de modificaciones incluyendo `parent` (0,25); eliminación lógica frente a física (0,25); argumento del sucesor sin hijo izquierdo y relevancia de 75 (0,25).
- c) 0,75: inorder correcto y por qué (0,25); invariante roto (0,25); efecto de `remove(35)` (0,25).
- d) 1,0: localización $O(h)$ (0,25); modificación $O(1)$ (0,25); traza de `remove(45)` (0,25); por qué no es $O(\log n)$ (0,25).

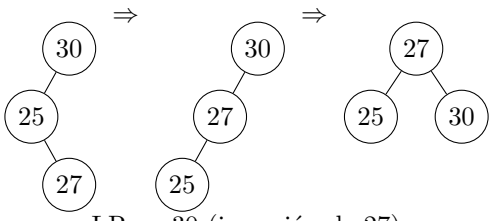
Solución del Problema 6

a) Inserciones (se indica `bf` = `balanceFactor`):

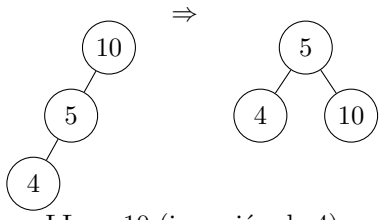
- `10`, `20`: sin desbalance.
- `30`: queda $10 \to 20 \to 30$ a la derecha. En 10: `bf` $= 0 - 2 = -2$. `bf(10.right = 20)` $= -1$, no es positivo: caso **RR**, `rotateLeft(10)`.
- `25`: `30.left = 25`. `bf(30)` $= 1$, `bf(20)` $= 1 - 2 = -1$. Sin desbalance.
- `27`: `25.right = 27`. `bf(25)` $= -1$; en 30: `bf` $= 2 - 0 = 2$. `bf(30.left = 25)` $= -1 < 0$: caso **LR**, `30.left = rotateLeft(25)` y luego `rotateRight(30)`.
- `5`: `10.left = 5`. `bf(10)` $= 1$, `bf(20)` $= 2 - 2 = 0$. Sin desbalance.
- `4`: `5.left = 4`. `bf(5)` $= 1$; en 10: `bf` $= 2 - 0 = 2$. `bf(10.left = 5)` $= 1 \geq 0$: caso **LL**, `rotateRight(10)`.
- `35`: `30.right = 35`. `bf(30)` $= -1$, `bf(27)` $= 1-2 = -1$, `bf(20)` $= 2-3 = -1$. Sin desbalance.
- `33`: `35.left = 33`. `bf(35)` $= 1$; en 30: `bf` $= 0 - 2 = -2$. `bf(30.right = 35)` $= 1 > 0$: caso **RL**, `30.right = rotateRight(35)` y luego `rotateLeft(30)`.



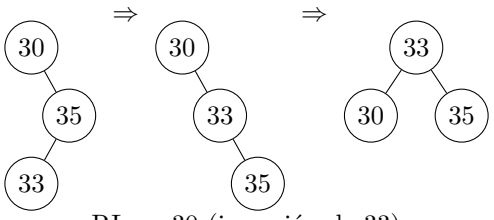
RR en 10 (inserción de 30)



LR en 30 (inserción de 27)

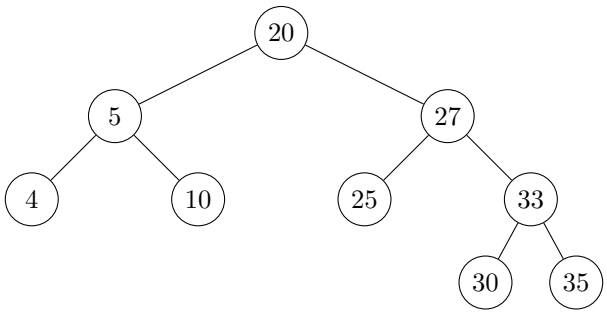


LL en 10 (inserción de 4)



RL en 30 (inserción de 33)

Árbol final:



Observación: en la inserción basta una sola reparación (simple o doble) por inserción, porque después de rotar el subárbol recupera la altura que tenía antes de insertar y los ancestros no cambian su factor de balance.

b) Inserción de 27. Antes del rebalanceo: `30.left = 25`, `25.right = 27`, alturas: 27 = 1, 25 = 2, 30 = 3.

`rotateLeft(25)` (con `x = 25`, `y = 27`, `middle = 27.left = null`):

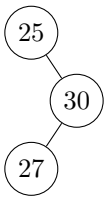
1. `27.left = 25` (`y.left = x`)
2. `25.right = null` (`x.right = middle`)
3. `updateHeight(25)` → 1; `updateHeight(27)` → 2; retorna 27.
4. En `rebalance`: `30.left = 27`.

`rotateRight(30)` (con `y = 30`, `x = 27`, `middle = 27.right = null`):

5. `27.right = 30` (`x.right = y`)
6. `30.left = null` (`y.left = middle`)
7. `updateHeight(30)` → 1; `updateHeight(27)` → 2; retorna 27.
8. Al volver de la recursión: `20.right = 27`, `updateHeight(20)` → 3 y `rebalance(20)` no rota.

Si en `rotateRight` se actualizara `x = 27` antes que `y = 30`, se usaría la altura vieja de 30, que es 3: `27.height` $= 1 + \text{máx}(1, 3) = 4$ en lugar de 2. Al subir, `20.height` sería 5 y `bf(20)` $= 1 - 4 = -3$, lo que provocaría una rotación innecesaria. La altura es información derivada: el padre debe calcularse con las alturas ya correctas de sus hijos, por eso primero se actualiza el nodo que quedó más abajo.

c) Versión defectuosa sobre `30.left = 25`, `25.right = 27`: `rotateRight(30)` toma `x = 25`, `middle = 25.right = 27`, asigna `25.right = 30` y `30.left = 27`.



Alturas: 27 = 1, 30 = 2, 25 = 3. `bf(25)` $= 0 - 2 = -2$. El subárbol sigue desbalanceado (solo cambió de lado), y además 20 tendría `bf` $= 1 - 3 = -2$. En el caso en zigzag, la rotación simple traslada el subárbol central (27) al otro lado sin reducir la altura; por eso primero hay que convertir el zigzag en línea recta con `rotateLeft(25)`.

d) Tabla del árbol final:

Nodo	height(left)	height(right)	height / balanceFactor
4	0	0	1 / 0
10	0	0	1 / 0
25	0	0	1 / 0
30	0	0	1 / 0
35	0	0	1 / 0
5	1	1	2 / 0
33	1	1	2 / 0
27	1	2	3 / −1
20	2	3	4 / −1

Todos los factores están en $\{-1, 0, 1\}$: se cumple el invariante AVL.

e) $N(1) = 1$, $N(2) = 2$, $N(3) = 1 + 2 + 1 = 4$, $N(4) = 1 + 4 + 2 = 7$, $N(5) = 1 + 7 + 4 = 12$.

Para tener altura 5 se necesitan al menos 12 nodos; con 9 nodos la altura máxima es 4 (y el árbol del problema alcanza justamente $h = 4$). Un BST sin balanceo con 9 nodos puede tener altura 9 (una cadena). $N(h)$ crece como la sucesión de Fibonacci, es decir, exponencialmente en h; por eso $h = O(\log n)$ en un AVL.

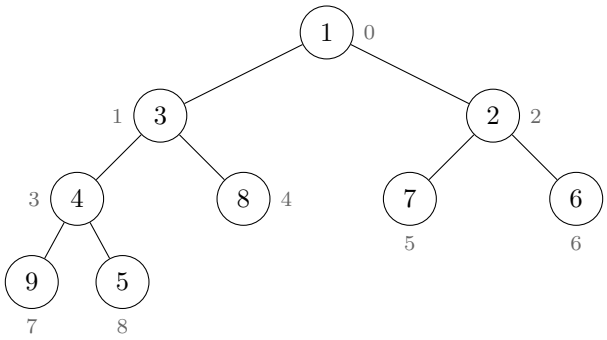
Criterio de corrección (4 puntos).

- a) 1,5: cada uno de los cuatro casos con nodo, nombre y rotaciones correctas (0,3 cada uno); árbol final (0,3).
- b) 0,75: referencias de ambas rotaciones en orden (0,25); orden de `updateHeight` (0,25); altura incorrecta calculada y su consecuencia (0,25).
- c) 0,75: resultado de la rotación simple (0,25); factores de balance que prueban la violación (0,25); explicación del zigzag (0,25).
- d) 0,5: alturas (0,25); factores de balance (0,25).
- e) 0,5: valores de $N(h)$ (0,25); altura máxima 4 frente a 9 (0,25).

Solución del Problema 7

a) `heapify()` con $n = 9$: `i` va desde $9/2 - 1 = 3$ hasta 0.

`i`	Intercambios de `trickleDown(i)`	Cantidad	Arreglo después
–	–	–	[9, 4, 7, 1, 8, 2, 6, 3, 5]
3	hijos `a[7]=3`, `a[8]=5`; $1 \leq 3$, sin cambio	0	[9, 4, 7, 1, 8, 2, 6, 3, 5]
2	hijos `a[5]=2`, `a[6]=6`; 7 con 2 (índice 5)	1	[9, 4, 2, 1, 8, 7, 6, 3, 5]
1	4 con 1 (índice 3); luego 4 con 3 (índice 7)	2	[9, 1, 2, 3, 8, 7, 6, 4, 5]
0	9 con 1 (índice 1); 9 con 3 (índice 3); 9 con 4 (índice 7)	3	[1, 3, 2, 4, 8, 7, 6, 9, 5]
	Total	**6**	



Heap final: `a =`

0	1	2	3	4	5	6	7	8
1	3	2	4	8	7	6	9	5

(los números pequeños junto a cada nodo son índices).

Verificación: $1 \leq 3, 2$; $3 \leq 4, 8$; $2 \leq 7, 6$; $4 \leq 9, 5$.

b) `remove()`: `min = 1`; `a[0] = a[8] = 5`; `a[8] = null`; $n = 8$. Arreglo: `[5, 3, 2, 4, 8, 7, 6, 9]`.

`trickleDown(0)`: hijos `a[1] = 3`, `a[2] = 2`; el menor es 2 (índice 2) y $2 < 5$: intercambio. Arreglo: `[2, 3, 5, 4, 8, 7, 6, 9]`. En `i = 2`: hijos `a[5] = 7`, `a[6] = 6`; el menor es 6 y $6 > 5$: se detiene.

Retorna **1**. Arreglo final: `[2, 3, 5, 4, 8, 7, 6, 9]`, $n = 8$.

c) `add(0)`: `a[8] = 0`, $n = 9$. `bubbleUp(8)`:

- `parent(8) = 3`: $0 < 4$, intercambio: `[2, 3, 5, 0, 8, 7, 6, 9, 4]`.
- `parent(3) = 1`: $0 < 3$, intercambio: `[2, 0, 5, 3, 8, 7, 6, 9, 4]`.
- `parent(1) = 0`: $0 < 2$, intercambio: `[0, 2, 5, 3, 8, 7, 6, 9, 4]`.
- `i = 0`: se detiene.

Arreglo final: `[0, 2, 5, 3, 8, 7, 6, 9, 4]`. Se realizaron 3 intercambios, uno por nivel (altura del heap: 4).

d) En `heapify()`, `trickleDown(i)` cuesta a lo más la altura del subárbol de `i` menos 1. Aproximadamente la mitad de los nodos son hojas (no se procesan), un cuarto tiene subárboles de altura 2 (a lo más 1 intercambio), un octavo de altura 3 (a lo más 2), etc.:

$$\frac{n}{4} \cdot 1 + \frac{n}{8} \cdot 2 + \frac{n}{16} \cdot 3 + \ldots \;=\; n \sum_{k \geq 1} \frac{k}{2^{k+1}} \;=\; n.$$

Muchos nodos baratos y pocos nodos caros: $O(n)$. En la parte a) se hicieron 6 intercambios con $n = 9$.

Con `add` uno por uno, el elemento i-ésimo puede subir hasta la raíz con `bubbleUp` de $O(\log i)$ intercambios (por ejemplo, si los valores llegan en orden decreciente cada nuevo elemento es el mínimo). El total es $\sum_{i=1}^{n} \log_2 i = \log_2(n!) \in \Theta(n \log n)$.

e) Estado de b) tras mover el último elemento a la raíz: `[5, 3, 2, 4, 8, 7, 6, 9]`. La versión del estudiante intercambia 5 con el hijo izquierdo 3 (porque $3 < 5$): `[3, 5, 2, 4, 8, 7, 6, 9]`; luego en `i = 1`, el hijo izquierdo 4 es menor que 5: `[3, 4, 2, 5, 8, 7, 6, 9]`.

Ahora la raíz 3 tiene como hijo derecho a 2: `a[parent(2)] = 3 > 2 = a[2]`, se viola el invariante, y `peek()` retornaría 3 aunque el mínimo es 2. Se debe elegir el **menor hijo válido**, porque solo él puede quedar como padre del otro hijo sin violar el orden.

f) La afirmación es falsa. El invariante del heap solo ordena verticalmente (padre menor o igual que cada hijo); no establece ninguna relación entre el subárbol izquierdo y el derecho. En el heap de a), el 9 está en el subárbol izquierdo y el 7 en el derecho; no hay forma de decidir hacia qué lado bajar para buscar un valor x. A diferencia de un BST, en cada nodo no se puede descartar un subárbol completo por comparación, por lo que la búsqueda cuesta $O(n)$ en el peor caso. La única poda posible (si `a[i] > x`, el subárbol de `i` no contiene a x) no mejora el peor caso. El heap es eficiente para `peek` ($O(1)$), `add` y `remove` ($O(\log n)$), no para búsqueda arbitraria.

Criterio de corrección (4 puntos).

- a) 1,25: orden de `i` desde 3 hasta 0 (0,25); intercambios correctos en `i = 2` e `i = 1` (0,25); cascada completa en `i = 0` (0,25); arreglo y árbol finales (0,25); total de intercambios (0,25).
- b) 0,75: valor retornado y movimiento del último a la raíz (0,25); elección del menor hijo (0,25); condición de parada y arreglo final (0,25).
- c) 0,5: pasos de `bubbleUp` con `parent(i)` (0,25); arreglo final (0,25).
- d) 0,5: suma acotada para `heapify` (0,25); justificación $O(n \log n)$ para inserciones sucesivas (0,25).
- e) 0,5: traza de la versión defectuosa (0,25); violación concreta del invariante (0,25).
- f) 0,5: ausencia de orden entre hermanos (0,25); conclusión $O(n)$ y contraste con BST (0,25).

Simulacro elaborado siguiendo el formato del Control 1 y la Práctica Calificada 1 de CC232 (2026-2). No es un examen oficial del curso.